

Name: Andy Bigley

Date: 26 November 2025

Course: IT FDN 130 A Au 25: Foundations of Databases & SQL Programming

Assignment07

Github URL: <https://github.com/abigley2004/DBFoundations-Module07>

Introduction

In Week 7, the focus expanded from working with views to designing and applying SQL User-Defined Functions (UDFs) to simplify recurring logic and support more expressive data retrieval. UDFs provided a structured way to encapsulate formatting rules, business calculations, and parameterized filtering that would otherwise require repetitive, error-prone SQL expressions. By building on the schemabound base views created in earlier weeks, UDFs introduced an additional abstraction layer that improved clarity, reduced duplication, and ensured consistent behavior across all queries and reporting tasks. This week's exercises demonstrated how scalar functions, inline table-valued functions, and multi-statement functions differ in purpose and performance, and how each contributes to a more maintainable and extensible database environment.

Academic Attribution Note:

I completed the coding and Q&A assignments independently after reviewing the course videos and materials. I used ChatGPT only to improve the clarity of my explanations and confirm the accuracy of my SQL concepts. The final work reflects my own understanding of the material.

1. Explain when you would use a SQL User-Defined Function.

A SQL User-Defined Function (UDF) is used when you need to encapsulate reusable logic that produces a value or a table from input parameters. UDFs are especially helpful when the same calculation, expression, or transformation appears in multiple queries, views, or stored procedures. Instead of rewriting the logic each time, a UDF provides a single, consistent definition that can be referenced anywhere in the database.

UDFs are used in situations where:

a. You need reusable calculations or formatting logic.

Functions are ideal for repeated expressions such as formatting dates, standardizing currency output, or computing a KPI (Key Performance Indicator) value. This promotes consistency across queries and reduces code duplication.

b. You want to simplify complex expressions.

A UDF can hide multi-step calculations or business rules behind a simple function call. This makes queries easier to read, debug, and maintain.

c. You need parameterized logic inside queries.

UDFs accept parameters, so they can return results tailored to user inputs, such as filtering by a product, date, or KPI (as done in Task 8 of the coding homework for this week).

d. You want to return a table from a reusable, parameterized query.

Inline table-valued functions (iTVFs) allow you to return a set of rows that can be joined to other tables. They are commonly used in reporting, filtering, and analytic scenarios where the same result set is produced repeatedly.

e. You need deterministic output for views.

Functions integrate cleanly into schemabound views and help enforce consistent behavior when views must follow strict structure or formatting rules.

2. Explain the differences between Scalar, Inline, and Multi-Statement Functions

SQL Server supports three primary types of user-defined functions, each designed for different purposes and performance characteristics. The key differences relate to what each type returns, how they are written, and how efficiently they execute.

Function Type	Returns	Logic Allowed	Best Use Case	Performance
Scalar	Single value	Single RETURN	Formatting, calculations, business rules	Moderate
Inline Table Value Function (TVF)	Table (single SELECT)	No procedural logic	Parameterized queries, joins	Best
Multi-Statement TVF	Table (table variable)	Multiple statements, procedural logic	Complex, multi-step calculations	Usually slowest

- a. A *scalar function* returns a single value—such as a number, string, or date. It accepts input parameters and performs a calculation, transformation, or formatting operation.

Characteristics

- Returns a single scalar value (e.g., INT, NVARCHAR, DATE).
 - Often used for formatting (e.g., converting a date to 'January, 2017'), mathematical calculations, or applying business rules.
 - Called inside **SELECT**, **WHERE**, or **ORDER BY** clauses.
 - May have performance overhead because they execute row-by-row but they differ in how they operate and how they are used within a system.
- b. An *inline table-valued function (iTVF)* returns the result of a single **SELECT** statement as a table. It behaves like a parameterized view and typically offers the best performance among TVFs.

Characteristics

- Returns a table defined directly by a single **SELECT** query.
 - Cannot contain procedural logic or multiple statements.
 - Highly efficient because SQL Server can inline and optimize the query plan.
 - Ideal for filtering, joining, or returning parameterized result sets.
- c. A *multi-statement TVF (MSTVF)* returns a table defined by a series of SQL statements. You declare a table variable, populate it with multiple operations, and return the result.

Characteristics

- Returns a table defined by the function body.
- Allows multiple steps: temporary storage, conditional logic, iteration, or complex transformations.
- More flexible than inline functions but often slower because:
 - They use table variables.
 - SQL Server cannot optimize them as aggressively.
- Useful when the logic cannot be expressed as a single **SELECT**.

To demonstrate how each function is defined and used, the following SQL examples are based on the material covered in this week's homework.

Function Type	SQL Code
Scalar	<pre> CREATE FUNCTION dbo.fnFormatMonthYear (@InputDate DATE) RETURNS NVARCHAR(30) AS BEGIN RETURN FORMAT(@InputDate, 'MMMM, yyyy'); END; GO </pre> What it does: This converts a raw date into a formatted string like 'January, 2017'. Usage: <pre> dbo.fnFormatMonthYear(@InputDate DATE) </pre>
Inline Table-Valued Functions (iTVFs)	<pre> CREATE FUNCTION dbo.fProductInventoriesWithPreviousMonthCountsWit hKPIs (@KPI INT -- must be -1, 0, or 1) RETURNS TABLE AS RETURN (SELECT TOP 100000 v.ProductName, v.InventoryDate, -- formatted 'January, 2017' v.InventoryCount, v.PreviousMonthCount, v.InventoryCountKPI FROM dbo.vProductInventoriesWithPreviousMonthCountsWit hKPIs AS v CROSS APPLY (SELECT dbo.fnMonthYearToDate(v.InventoryDate) AS InventoryDateKey </pre>

Function Type	SQL Code
Inline Table-Valued Functions (iTVFs)	<pre>) AS d WHERE v.InventoryCountKPI = @KPI -- use parameter here ORDER BY v.ProductName, d.InventoryDateKey -- true chronological order); </pre> What it does: Returns a <i>table</i> of product inventory values filtered by a Key Performance Indicator (KPI) parameter. Usage: dbo.fProductInventoriesWithPreviousMonthCountsWit hKPIs(@KPI INT)
Multi-Statement Table-Valued Functions (MSTVFs)	<pre> CREATE OR ALTER FUNCTION dbo.fProductInventoryRunningChanges (@ProductName NVARCHAR(100)) RETURNS @Results TABLE (InventoryDate DATE, InventoryCount INT, PreviousMonthCount INT, ChangeFromPrevious INT, RunningTotalChange INT) AS BEGIN DECLARE @PrevCount INT = NULL; DECLARE @Running INT = 0; -- Step 1: Insert base rows (date + counts) ordered chronologically INSERT INTO @Results (InventoryDate, InventoryCount) SELECT I.InventoryDate, I.InventoryCount </pre>

Function Type	SQL Code
Multi-Statement Table-Valued Functions (MSTVFs)	<pre> FROM dbo.vProductInventoriesWithPreviousMonthCounts AS I WHERE I.ProductName = @ProductName ORDER BY I.InventoryDate; -- Step 2: Cursor-like row-by-row update for previous month logic UPDATE R SET PreviousMonthCount = @PrevCount, ChangeFromPrevious = CASE WHEN @PrevCount IS NULL THEN NULL ELSE R.InventoryCount - @PrevCount END, @Running = @Running + CASE WHEN @PrevCount IS NULL THEN 0 ELSE R.InventoryCount - @PrevCount END, RunningTotalChange = @Running, @PrevCount = R.InventoryCount FROM @Results AS R ORDER BY R.InventoryDate; RETURN; END; GO </pre> What it does: It loads all inventory months for the product into the result table in chronological order and returns running month-over-month inventory changes for a single product across all inventory months. Usage: <pre> SELECT * FROM dbo.fProductInventoryRunningChanges('Chai') ORDER BY InventoryDate </pre>

Together, these functions help developers simplify queries, enforce consistent logic, and organize database operations in a maintainable way.

Summary

Overall, this week highlighted the value of UDFs in creating modular, reusable logic that enriches both simple and complex SQL queries. Scalar functions streamlined operations such as formatting date values and managing edge cases in previous-month calculations. Inline table-valued functions enabled parameterized result sets that integrated seamlessly with downstream analytic views, including those incorporating KPIs and month-over-month inventory comparisons. Through these tasks, the role of UDFs as an essential component of an abstraction layer became clear: they promote consistency, enhance readability, and support scalable reporting design. With these foundational skills established, the transition to more advanced SQL topics will build on a robust understanding of how functions complement views, joins, and data-modeling practices to produce more accurate and maintainable data systems.